D

TAREQO

01 — Vue d'ensemble

Version 2.1.3+23 · 14 septembre 2026

TAREQO

Table des matières

Qu'est-ce que TAREQO ERP ? 3

Objectifs du produit 3

Fonctionnalités 3

- Implémentées (v1.0.0) 3
- Points forts du design 4

Public cible et plateformes 4

Stack et dépendances 5

État actuel et prochaines étapes 5

Documents connexes 6

Qu'est-ce que TAREQO ERP ?

TAREQO ERP est un système de gestion destiné aux petits commerçants. Il réunit les opérations quotidiennes d'un magasin dans une seule app :

- Stock — enregistrement et contrôle des produits avec lots FIFO et péremption (achat, vente, quantité disponible, péremption) ;
- Ventes — enregistrement des ventes avec panier, moyen de paiement et déduction automatique du stock ;
- Finances — flux de trésorerie (entrées × sorties), transactions et montants en attente ;
- Crédit (« fiado ») — suivi de qui doit au commerçant, de l'échéance et de l'historique des règlements.

TAREQO est le surnom du projet ; le produit lui-même est un ERP léger axé sur la convivialité et la simplicité.

Objectifs du produit

1. Répondre aux questions essentielles du commerçant :
2. « Combien ai-je vendu ? » (total des ventes) ;
3. « Quelle trésorerie ai-je ? » (solde du flux de caisse) ;
4. « Combien me doit-on à crédit, et qui ? » (à recevoir) ;
5. « Quel stock ai-je et qu'est-ce qui manque ? » (stock faible).
6. Éviter le « trou noir du crédit » : transformer le crédit accordé en un chiffre visible et gérable (qui doit, combien, quand).
7. Garantir l'intégrité de l'entreprise : stock jamais négatif, ventes atomiques, caisse cohérente.
8. Être un modèle SaaS multi-tenant : authentification par magasin/commerçant (Supabase), prêt pour le RLS (Row Level Security).

Fonctionnalités

Implémentées (v1.0.0)

Module	Fonctionnalités
Authentification	Inscription et connexion par e-mail/mot de passe (Supabase), session persistante restaurée au démarrage, déconnexion
Accueil (Dashboard)	KPI (total des ventes, solde de caisse, crédits à recevoir, stock), actions rapides, ventes récentes, alerte de stock faible
Produits	Liste avec recherche locale, inscription via bottom sheet (avec validation des prix), code-barres de balance/PLU et vente au poids (W11), pull-to-refresh, chip de stock faible

Stock (FIFO, W11)	Lots par entrée de stock avec quantité, péremption et coût d'acquisition ; diminution FIFO au PDV (péremption la plus proche d'abord) ; alertes de péremption sur le tableau de bord (rouge = périmé, jaune = < 30 jours)
Ventes	Historique des ventes, panier de nouvelle vente (steppers de quantité et fractions décimales pour les produits au poids), lecture du code-barres d'étiquette (`20PPPPPWWWWW`), 6 moyens de paiement, détail de vente avec snapshots de prix
Achats	Entrée de marchandise fournisseur : panier avec steppers, péremption facultative par article (crée les lots de l'entrée), augmente le stock et enregistre la dépense en caisse (catégorie « Achats »)
Finances	Flux de trésorerie par période (Aujourd'hui / 7 jours / Ce mois-ci / Tout), entrées/sorties/solde, enregistrement de transaction, montants en attente avec bouton « Recevoir »
Crédit (« fiado »)	Total à recevoir, dettes par échéance (chips de retard), enregistrement de crédit éventuellement lié à un client, règlement avec confirmation, onglet d'historique des règlements
Clients	Inscription (nom, téléphone, e-mail), recherche, total de crédit en attente par client sur la carte
Rapports	Chiffre d'affaires, bénéfice brut, marge %, meilleures ventes (top 5) et ventes par jour, avec sélecteur de période
Comptes (SaaS)	Le propriétaire s'inscrit, choisit un forfait et attend l'approbation du super administrateur (≤ 24 h) ; une fois approuvé, il crée des magasins et des employés (avec limites du forfait) et demande des évolutions de forfait
Super Administrateur	Panneau d'administration : approuver/rejeter les demandes de tenant, décider des évolutions de forfait et créer/modifier les forfaits (qui limitent magasins et employés de chaque tenant)
Multi-magasins (Phase 2)	Portée par magasin (store_id) sur produits, ventes, finances, crédit, clients et achats — chaque magasin avec son propre stock, sa caisse et son historique — avec un sélecteur de magasin actif en haut de chaque écran

Points forts du design

- Navigation adaptative : NavigationRail sur écrans larges (web/desktop) et NavigationBar sur écrans étroits (mobile), avec 6 onglets (Accueil, Produits, Ventes, Finances, Crédit, Clients) et Achats/Rapports/Magasins accessibles depuis le tableau de bord (section « Modules »).
- Accès par rôle : le super administrateur entre dans le panneau d'administration ; un propriétaire non approuvé voit des écrans d'attente/motif jusqu'à la libération ; un propriétaire approuvé opère l'ERP et gère magasins/employés.
- Thèmes clair et sombre (Material 3) avec un seul accent vert émeraude.
- Formatage fr-BR : devise (R$ 1.234,56), dates (04/08/2026) et jours relatifs (Hoje, Ontem).
- Données 100 % réelles : l'app ne contient aucune donnée de démonstration — chaque tenant démarre vide et n'enregistre que ce que l'utilisateur crée.

Public cible et plateformes

- Utilisateur final : petits commerçants (épiceries, papeteries, magasins généraux, boutiques de quartier) qui doivent contrôler stock, ventes et crédit.
- Plateformes : Android, iOS, Windows, Linux, macOS et Web (avec repli sur persistance en mémoire).

Stack et dépendances

Package	Rôle	Couche
flutter_riverpod ^3.4.2	Gestion d'état réactive (AsyncNotifier)	présentation
get_it ^9.2.1	Injection de dépendances (Service Locator)	core
freezed 3.2.x + json_serializable	Entités immuables et sérialisation JSON (codegen)	domaine/données
supabase_flutter ^2.16.0	Authentification SaaS et futur backend (Postgres/RLS)	données
flutter_secure_storage ^10.3.1	Persistance chiffrée (Keychain/Keystore)	core
sqlite3 + sqlite3_flutter_libs	Persistance relationnelle locale (SQL paramétré)	données
mobile_scanner	Lecture code-barres par caméra au PDV (balances/PLU, W11)	présentation
path_provider + path	Emplacement du fichier de base de données	core/données
sentry_flutter ^9.26.0	Signalement de crash (Sentry, DSN via --dart-define=SENTRY_DSN)	core
flutter_localizations + intl	i18n (pt/en/fr) et formatage devise/date	core

> Dart SDK ^3.12.2 (Flutter 3.x récent).

État actuel et prochaines étapes

L'app est entièrement fonctionnelle de bout en bout. Avec Supabase configuré (par défaut), les modules métier se synchronisent sur Postgres par tenant et par magasin — produits, ventes, finances, crédit, clients et achats — avec RLS + triggers d'isolation (schéma dans supabase/schema.sql), et les flux d'argent (vente, achat, dette, règlement) s'exécutent comme RPC atomiques `SECURITY DEFINER` dans une seule transaction de base de données — y compris la diminution des lots FIFO lors des ventes et la création de lots lors des achats (W11). Sans Supabase configuré, la persistance retombe sur SQLite (natif) ou en mémoire (web).

Feuille de route restante :

1. Dépenses fixes récurrentes (planification du loyer/électricité) ;
2. Action « Payer » pour les dépenses en attente dans Finances (aujourd'hui seules les entrées ont « Recevoir ») ;
3. Cache hors ligne (lecture locale + synchronisation) pour une utilisation sans connexion ;
4. Agrégation/pagination côté serveur pour les grands volumes (les KPI sont calculés côté client aujourd'hui, plafonnés à 1000 lignes par liste) ;
5. Facturation/paiements et changements de forfait en libre-service.

Documents connexes

- Architecture — comment le code est organisé.
- Flux métier — comment fonctionnent ventes, crédit et caisse en interne.
- Multi-tenant et Super Admin — forfaits, approbation d'accès et gestion des magasins.
- Guide du développeur — comment exécuter et faire évoluer le projet.